

11e — Service Mesh

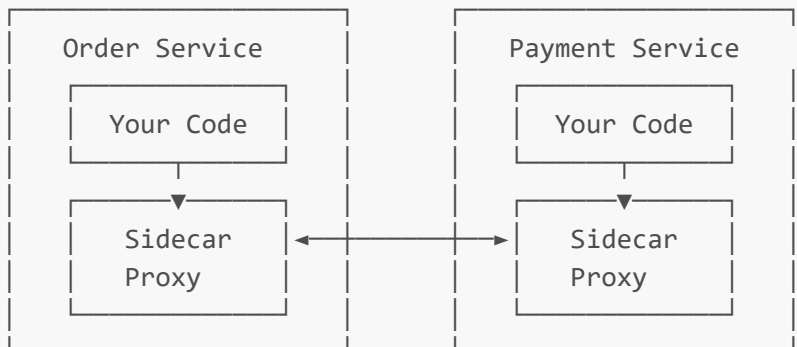
Key Concepts

- **Service mesh** — infrastructure layer that handles cross-cutting concerns in service-to-service communication without changing application code
- **Sidecar proxy** — separate proxy process running alongside each service; intercepts all traffic
- **Control plane** — central brain that configures all sidecars (routing rules, retry policies, security)
- **Data plane** — the actual sidecar proxies that handle traffic
- **mTLS** — mutual TLS; encrypts all service-to-service traffic automatically
- **East-West traffic** — service-to-service communication inside the system
- **North-South traffic** — external client traffic coming into the system

The Problem It Solves

- 20 services each need: retries, timeouts, mTLS, load balancing, logging
- Without mesh → every team implements this separately → **duplication + inconsistency**
- With mesh → one infrastructure layer handles it for everyone → **no application code changes**

How It Works — Sidecar Pattern



- Your code talks to its **local sidecar**, not directly to other services
- Sidecar handles everything transparently — your code doesn't know it exists

Control Plane vs Data Plane

Control Plane (e.g. Istiod)

- └─ Configures sidecar in Order Service
- └─ Configures sidecar in Payment Service
- └─ Configures sidecar in Notification Service

Data Plane (Envoy sidecars)

- └─ Actually handles all traffic between services

What the Sidecar Handles

Responsibility	What It Does
Retries	Automatically retries failed requests
Timeouts	Cuts off slow requests after threshold
Load balancing	Distributes traffic across instances
mTLS	Encrypts all service-to-service traffic
Circuit breaking	Stops calling a failing service
Observability	Captures metrics, traces, logs per request

Service Mesh vs API Gateway

	API Gateway	Service Mesh
Traffic direction	North-South (client → services)	East-West (service → service)
Purpose	Entry point for external traffic	Internal service communication
Examples	Kong, AWS API Gateway, Azure APIM	Istio, Linkerd

Popular Tools

Tool	Notes
Istio	Most popular; uses Envoy as sidecar
Linkerd	Lightweight, simpler than Istio
Consul Connect	HashiCorp; good for hybrid cloud
AWS App Mesh	AWS-native; uses Envoy

When to Use / Avoid

Use When	Avoid When
10+ microservices	Small number of services
Cross-cutting concerns becoming repetitive	Simple apps — adds complexity for little gain
Need encryption between all services	Small teams without operational capacity
Need deep observability across service calls	

Industry Examples

- **Uber** — Istio manages communication between 2000+ microservices; consistent retry/timeout across all teams
- **Airbnb** — automatic mTLS between services; security enforced without touching application code

Interview Questions

Q: What is a service mesh? A: An infrastructure layer that handles cross-cutting concerns — retries, timeouts, load balancing, mTLS — between services using sidecar proxies, without touching application code.

Q: What is the sidecar pattern? A: A separate proxy process runs alongside each service. All traffic goes through the sidecar, which handles retries, mTLS, and observability transparently.

Q: What's the difference between a service mesh and an API gateway? A: API gateway handles north-south traffic — external clients into your system. Service mesh handles east-west traffic — communication between internal services.

Q: When would you use a service mesh? A: When you have 10+ services and cross-cutting concerns like retries, timeouts, and encryption are becoming inconsistent across teams.